

DLX Assembly Language

Dr. Jonathan Phillips

Utah State University

ECE 6740 – Advanced Reconfigurable Computing

Overview

- DLX Processor
 - RISC Architecture
 - Designed by John Hennessy and David Patterson
 - Designed for university courses
 - 5-stage pipeline
 - 32 general-purpose registers
- We are going to implement a modified super-subset of DLX

General DLX syntax:

ADDI R1, R0, 0

ADDI R2, R0, 1

LW R3, n(R0)

SW result(R0), R3

J done

NOP

Destination, Source1, Source2

Base + Offset

Labels

DLX Instruction Types

- Register
 - All operands are registers

3	3	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0
opcode					rd					rs1					rs2					unused											

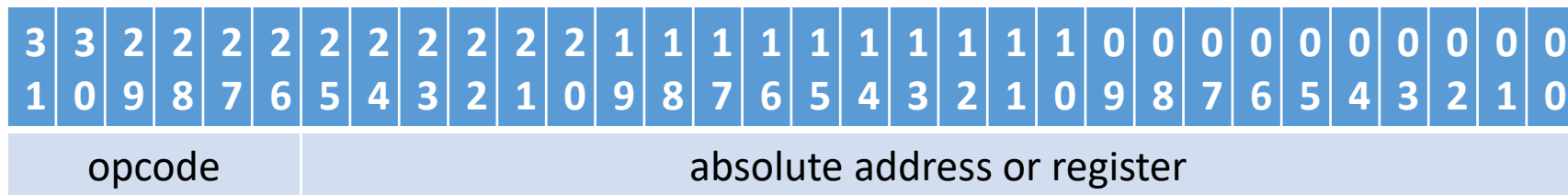
DLX Instruction Types

- Immediate
 - At least one operand is an immediate value

3	3	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0
opcode						rd						rs1				immediate															

DLX Instruction Types

- Jump
 - Modifies program counter
 - Our version will use absolute addresses; original DLX uses relative addresses
 - Pros / Cons?



DLX Instruction Types

- Memory

- Load data into register from memory
- Store data from register into memory

3	3	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0		
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0
opcode					r_data					r_offset					base_address																

DLX Instruction Set

Mnemonic	Description	Meaning	Opcode (6 bits)
NOP	No Operation	Do nothing	0x00
LW r_data, base(r_offset)	Load Word	$r_data = \{base + r_offset\}$	0x01
SW base(r_offset), r_data	Store Word	$\{base + r_offset\} = r_data$	0x02

DLX Instruction Set

Mnemonic	Description	Meaning	Opcode (6 bits)
ADD rd, rs1, rs2	Add (signed)	$rd = rs1 + rs2$	0x03
ADDI rd, rs1, imm	Add Immediate (signed)	$rd = rs1 + imm$	0x04
ADDU rd, rd1, rs2	Add (unsigned)	$rd = rs1 + rs2$	0x05
ADDUI rd, rs1, imm	Add Immediate (unsigned)	$rd = rs1 + imm$	0x06
SUB rd, rs1, rs2	Subtract (signed)	$rd = rs1 - rs2$	0x07
SUBI rd, rs1, imm	Subtract Immediate (signed)	$rd = rs1 - imm$	0x08
SUBU rd, rd1, rs2	Subtract (unsigned)	$rd = rs1 - rs2$	0x09
SUBUI rd, rs1, imm	Subtract Immediate (unsigned)	$rd = rs1 - imm$	0x0A

DLX Instruction Set

Mnemonic	Description	Meaning	Opcode (6 bits)
AND rd, rs1, rs2	And	$rd = rs1 \& rs2$	0x0B
ANDI rd, rs1, imm	And Immediate	$rd = rs1 \& imm$	0x0C
OR rd, rs1, rs2	Or	$rd = rs1 \mid rs2$	0x0D
ORI rd, rs1, imm	Or Immediate	$rd = rs1 \mid imm$	0x0E
XOR rd, rs1, rs2	Exclusive Or	$rd = rs1 \wedge rs2$	0x0F
XORI rd, rs1, imm	Exclusive Or Immediate	$rd = rs1 \wedge imm$	0x10

DLX Instruction Set

Mnemonic	Description	Meaning	Opcode (6 bits)
SLL rd, rs1, rs2	Shift Left Logical	$rd = rs1 \ll rs2$	0x11
SLLI rd, rs1, imm	Shift Left Logical Immediate	$rd = rs1 \ll imm$	0x12
SRL rd, rs1, rs2	Shift Right Logical	$rd = rs1 \gg rs2$ (fill with 0)	0x13
SRLI rd, rs1, imm	Shift Right Logical Immediate	$rd = rs1 \gg imm$ (fill with 0)	0x14
SRA rd, rs1, rs2	Shift Right Arithmetic	$rd = rs1 \gg rs2$ (sign extend)	0x15
SRAI rd, rs1, imm	Shift Right Arithmetic Immediate	$rd = rs1 \gg imm$ (sign extend)	0x16

DLX Instruction Set

Mnemonic	Description	Meaning	Opcode (6 bits)
SLT rd, rs1, rs2	Set If Less Than (signed)	rd = 1 if rs1 < rs2; else 0	0x17
SLTI rd, rs1, imm	Set If Less Than Immediate (signed)	rd = 1 if rs1 < imm; else 0	0x18
SLTU rd, rs1, rs2	Set If Less Than (unsigned)	rd = 1 if rs1 < rs2; else 0	0x19
SLTUI rd, rs1, imm	Set If Less Than Immediate (unsigned)	rd = 1 if rs1 < imm; else 0	0x1A
SGT rd, rs1, rs2	Set If Greater Than (signed)	rd = 1 if rs1 > rs2; else 0	0x1B
SGTI rd, rs1, imm	Set If Greater Than Immediate (signed)	rd = 1 if rs1 > imm; else 0	0x1C
SGTU rd, rs1, rs2	Set If Greater Than (unsigned)	rd = 1 if rs1 > rs2; else 0	0x1D
SGTUI rd, rs1, imm	Set If Greater Than Immediate (unsigned)	rd = 1 if rs1 > imm; else 0	0x1E

DLX Instruction Set

Mnemonic	Description	Meaning	Opcode (6 bits)
SLE rd, rs1, rs2	Set If Less Than or Equal (signed)	rd = 1 if $rs1 \leq rs2$; else 0	0x1F
SLEI rd, rs1, imm	Set If Less Than or Equal Immediate (signed)	rd = 1 if $rs1 \leq imm$; else 0	0x20
SLEU rd, rs1, rs2	Set If Less Than or Equal (unsigned)	rd = 1 if $rs1 \leq rs2$; else 0	0x21
SLEUI rd, rs1, imm	Set If Less Than or Equal Immediate (unsigned)	rd = 1 if $rs1 \leq imm$; else 0	0x22
SGE rd, rs1, rs2	Set If Greater Than or Equal (signed)	rd = 1 if $rs1 \geq rs2$; else 0	0x23
SGEI rd, rs1, imm	Set If Greater Than or Equal Immediate (signed)	rd = 1 if $rs1 \geq imm$; else 0	0x24
SGEU rd, rs1, rs2	Set If Greater Than or Equal (unsigned)	rd = 1 if $rs1 \geq rs2$; else 0	0x25
SGEUI rd, rs1, imm	Set If Greater Than or Equal Immediate (unsigned)	rd = 1 if $rs1 \geq imm$; else 0	0x26
SEQ rd, rs1, rs2	Set If Equal	rd = 1 if $rs1 = rs2$; else 0	0x27
SEQI rd, rs1, imm	Set If Equal Immediate	rd = 1 if $rs1 = imm$; else 0	0x28
SNE rd, rs1, rs2	Set If Not Equal	rd = 1 if $rs1 \neq rs2$; else 0	0x29
SNEI rd, rs1, imm	Set If Not Equal Immediate	rd = 1 if $rs1 \neq imm$; else 0	0x2A

DLX Instruction Set

Mnemonic	Description	Meaning	Opcode (6 bits)
BEQZ rs1, label	Branch If Equal to Zero	pc = label if rs1 = 0	0x2B
BNEZ rs1, label	Branch If Not Equal to Zero	pc = label if rs1 $\neq$ 0	0x2C
J label	Jump	pc = label	0x2D
JR rs1	Jump (Register)	pc = rs1	0x2E
JAL label	Jump And Link	r31 = pc, pc = label	0x2F
JALR rs1	Jump And Link (Register)	r31 = pc, pc = rs1	0x30

Example DLX Program

```
;Comments always consume entire lines
```

```
.data
```

```
    n          1          5  
    result     1          0
```

```
.text
```

```
    ADDI       R1, R0, 0  
    ADDI       R2, R0, 1  
    LW         R10, n(R0)
```

```
top
```

```
    SLEI       R11, R10, 1  
    BNEZ       R11, exit  
    ADD        R3, R1, R2  
    ADDI       R1, R2, 0  
    ADDI       R2, R3, 0  
    SUBI       R10, R10, 1  
    J          top
```

```
exit
```

```
    SW         result(R0), R3
```

```
done
```

```
    NOP  
    J          done
```

What does this
program do?

Memory Initialization File (.mif)

- File format for specifying memory contents
- Supported by Intel Quartus Prime for memory initialization
- https://www.intel.com/content/www/us/en/programmable/quartus_help/18.1/index.htm#reference/glossary/def_mif.htm

Creating Machine Code:

- DLX assembly file(s) →
 - Binary data segment file
 - Binary code segment file

```

;This is a comment
;Comments always consume entire lines

.data

    n        1        10
    result   1        0

.text

    ADDI     R1, R0, 0
    ADDI     R2, R0, 1
    LW       R10, n(R0)

top

    SLEI     R11, R10, 1
    BNEZ     R11, exit
    ADD      R3, R1, R2
    ADDI     R1, R2, 0
    ADDI     R2, R3, 0
    SUBI     R10, R10, 1
    J        top

exit

    SW       result(R0), R3

done

    J        done

```



```

DEPTH = 1024;
WIDTH = 32;
ADDRESS_RADIX = HEX;
DATA_RADIX = HEX;
CONTENT
BEGIN

    000 : 0000000A;          --n[0]
    001 : 00000000;          --result[0]

END;

```

```

;This is a comment
;Comments always consume entire lines

.data

    n      1      10
    result 1      0

.text

    ADDI    R1, R0, 0
    ADDI    R2, R0, 1
    LW      R10, n(R0)

top

    SLEI    R11, R10, 1
    BNEZ    R11, exit
    ADD     R3, R1, R2
    ADDI    R1, R2, 0
    ADDI    R2, R3, 0
    SUBI    R10, R10, 1
    J       top

exit

    SW      result(R0), R3

done

    J       done

```



```

DEPTH = 1024;
WIDTH = 32;
ADDRESS_RADIX = HEX;
DATA_RADIX = HEX;
CONTENT
BEGIN

000 : 10200000;      --ADDI    R1, R0, 0
001 : 10400001;      --ADDI    R2, R0, 1
002 : 05400000;      --LW      R10, n(R0)
003 : 816A0001;      --SLEI    R11, R10, 1
004 : B160000A;      --BNEZ    R11, 00A
005 : 0C611000;      --ADD     R3, R1, R2
006 : 10220000;      --ADDI    R1, R2, 0
007 : 10430000;      --ADDI    R2, R3, 0
008 : 214A0001;      --SUBI    R10, R10, 1
009 : B4000003;      --J       003
00A : 08600001;      --SW      result(R0), R3
00B : B400000B;      --J       00B

END;

```

Observations

- Code and Data segments are separated because we are implementing a Harvard architecture
- Labels (in both code and data segments) need to be converted to absolute addresses
- How do we handle a label that is referenced before it is defined?